TD1 - PRAM

Organization. The exercises that we cannot cover in class are homework: their solutions will be posted, and we may go over some of them later in class. The algorithms of an exercise may be used in the following exercises.

Conventions. Arrays are indexed from 1, Nil denotes the null pointer, and cx denotes a copy of the array x. The *time* of an algorithm is its parallel running time on the PRAM, using the number of PUs it is given. Its *work*, cost, work-optimality and efficiency are defined as in the lecture, with respect to the time T_{seq} of the best sequential algorithm for the same problem. The following algorithms of the lecture may be called as building blocks; they all run on an EREW PRAM with N PUs.

Algorithm	Returns	Time	Work	Cost
Bcast(e, N)	$pe[1 \dots N]$, with $pe[i] = e$ for all i	$O(\log_2 N)$	$O(N)$	$O(N \log_2 N)$
Reduce$(A[1 \dots N], \otimes)$	$A[1] \otimes \cdots \otimes A[N]$, for an associative $\otimes$	$O(\log_2 N)$	$O(N)$	$O(N \log_2 N)$
RankComputation$(next[1 \dots N])$	$d[1 \dots N]$, the distances to the end of the list	$O(\log_2 N)$	$O(N \log_2 N)$	$O(N \log_2 N)$

Exercise 1

Is the array sorted?

Let $A[1 \dots N]$ be an array of N integers.

a) Design an algorithm IsSorted$(A[1 \dots N])$ that returns *true* if $A[1] \leq A[2] \leq \cdots \leq A[N]$ and *false* otherwise, in $O(1)$ time on a CRCW PRAM with N PUs. List all the concurrency modes that can be employed for the writes of this algorithm.

b) Verify the reads of your algorithm, by decomposing them into elementary steps as was done for list ranking in the lecture: are they exclusive? Does the algorithm need concurrent reads, or only concurrent writes?

c) Design an EREW PRAM version of IsSorted with N PUs. What are its time, work and cost? Is it work-optimal? Is it efficient?

Exercise 2

From a list to a ring

Let $next[1 \dots N]$ be a linked list of N nodes: $next[i]$ is the successor of node i, and $next[\ell] =$ Nil for the last node ℓ. The head h of the list is the only node that no other node points to. We want to close the list into a ring, by making the last node point to the head: $next[\ell] \leftarrow h$.

a) Design an EREW PRAM algorithm ListToRing$(next[1 \dots N])$ with N PUs that updates $next[1 \dots N]$ into a ring and returns the index h of the head. Verify that all reads and writes are exclusive.

b) What are its time, work and cost? Is it work-optimal? Is it efficient?

Exercise 3

Inverting a list

Let $next[1 \dots N]$ be a linked list of N nodes.

a) Design an EREW PRAM algorithm Invert$(next[1 \dots N])$ with N PUs that computes the inverse of this linked list, and returns it in a pointer array $prev[1 \dots N]$ (e.g., if $next[3] = 7$, then $prev[7] = 3$).

b) What are its time, work and cost? Is it work-optimal? Is it efficient?

c) Now let $next[1 \dots N]$ be a ring (no pointer is Nil), and h the index of one of its nodes. Design an EREW PRAM algorithm RingToList$(next[1 \dots N], h)$ with N PUs that turns the ring into a list whose head is h. What is its time?

Exercise 4

Work-optimal and efficient broadcast and reduction

Let e be a value, $A[1 \dots N]$ an array and $\otimes$ an associative operation. In the lecture, $\textsc{Bcast}(e, N)$ returned $pe[1 \dots N]$ with $pe[i] = e$ for all i, and $\textsc{Reduce}(A[1 \dots N], \otimes)$ computed $A[1] \otimes \cdots \otimes A[N]$, both in $O(\log_2 N)$ time on an EREW PRAM with N PUs, whereas $T_{seq}(N) = O(N)$ for both problems.

a) The Bcast of the lecture is *iterative* (a loop on k, the number of doublings done so far) and creates N PUs at every iteration. Modify it, keeping the name $\textsc{Bcast}(e, N)$, so that it uses up to N PUs, still runs in $O(\log_2 N)$ time, and becomes work-optimal. Is it efficient?

b) Write a *recursive* version $\textsc{BcastRec}(e, N)$ of the broadcast that uses up to N PUs, runs in $O(\log_2 N)$ time, and is work-optimal. Is it efficient?

c) Design $\textsc{BcastP}(e, N, P)$ that does the same broadcast with P PUs, $P \ll N$. What are its time, work and cost, as functions of N and P? For which values of P does the algorithm stay efficient?

d) Write an *iterative* version $\textsc{Reduce}(A[1 \dots N], \otimes)$ of the reduction that uses up to N PUs, runs in $O(\log_2 N)$ time, and is work-optimal. Is it efficient?

e) The reduction of the lecture is *recursive*: write it as $\textsc{ReduceRec}(A[1 \dots N], \otimes)$. Is it work-optimal? Is it efficient? Which PUs does it create, compared with your iterative version?

f) Design $\textsc{ReduceP}(A[1 \dots N], \otimes, P)$ that computes the same reduction with P PUs, $P \ll N$. What are its time, work and cost, as functions of N and P? For which values of P does the algorithm stay efficient?

Exercise 5

Prefix scan on an array

Let $X[1 \dots N]$ be an array of size N and $\otimes$ an associative operation.

a) Design an EREW PRAM algorithm $\textsc{Scan}(X[1 \dots N], \otimes)$ that computes and returns the prefix scan array $X'[i] = X[1] \otimes \cdots \otimes X[i]$ for all i, using N PUs and running in $O(\log_2 N)$ time. Verify that all reads and writes are exclusive. Hint: Try using recursion on the array. Compute the work and cost of this algorithm. Is it work-optimal? Is it efficient?

b) This time, design an algorithm specifically for P PUs that does the same computation in $O(N/P + \log_2 P)$ time, then compute its work and cost. Is it work-optimal? For what values of P does it stay efficient?

Exercise 6

Scan on a list

Let $next[1 \dots N]$ be a linked list of N nodes, with head h and tail t ($next[t] = \textsc{Nil}$), $x[1 \dots N]$ a value attached to each node, and $\otimes$ an associative operation. In the lecture, the list ranking algorithm computed, with pointer jumping, the distance $d[i]$ from every node i to the end of the list.

a) Design an algorithm $\textsc{Scan}(next[1 \dots N], x[1 \dots N], \otimes)$ that computes and returns the array $y[1 \dots N]$ where $y[i] = x[h] \otimes \cdots \otimes x[i]$ is the combination of the values from the head h of the list to node i, in $O(\log_2 N)$ time using N PUs. Hint: start from the list ranking algorithm of the lecture.

b) Verify the memory accesses of your algorithm: are all reads exclusive? all writes? On which PRAM model does it run as written, and how would you modify it to run on an EREW PRAM?

c) Compute the work and cost of this algorithm. Is it work-optimal? Is it efficient?

d) Show that computing the rank $r[i]$ of every node from the head of the list (0 for the head, $N - 1$ for the tail) is a particular case of Scan: for which values $x[i]$ and which operation $\otimes$?

e) Using a single call to Scan, design an algorithm $\textsc{RankByPrefix}(next[1 \dots N])$ with N PUs that computes and returns the distance $d[i]$ from every node to the end of the list.

f) Design an algorithm $\textsc{ReverseScan}(next[1 \dots N], x[1 \dots N], \otimes)$ that computes, in the same time, the array $y[1 \dots N]$ where $y[i] = x[i] \otimes \cdots \otimes x[t]$ is the combination of the values from node i to the tail t of the list. Pay attention to the order of the operands if $\otimes$ is not commutative.

Exercise 7

Selection in a list on an EREW PRAM

Let $next[1 \dots N]$ be a linked list of N nodes, each node i having a label $label[i] \in \{\text{red}, \text{blue}\}$.

a) Design an EREW PRAM algorithm SelectBlues($next[1 \dots N], label[1 \dots N]$) with N PUs that updates the list $next[1 \dots N]$ so that in the end i) each blue node points to the next blue node in the original list, ii) the last blue node and all the red nodes point to Nil, and iii) the algorithm returns the index of the first blue node, the head of the updated list of blue nodes (Nil if there is no blue node).

b) What are its time, work and cost?

Exercise 8

Separation in an array on an EREW PRAM

Let $A[1 \dots N]$ be an array of N objects, each element having a label Label($A[i]$) $\in \{0, 1\}$.

a) Design an EREW PRAM algorithm Separate01($A[1 \dots N]$) with N PUs that returns a copy $A'[1 \dots N]$ of A permuted so that all the elements of label 0 come first (the labels of $A'[1], \dots, A'[N]$ read $0, \dots, 0, 1, \dots, 1$). Hint: Try to use the prefix scan algorithm to determine where to write each element after permutation.

b) What are its time, work and cost?

Exercise 9

Sorting an array of unique integers

Let $X[1 \dots N]$ be an array of unique integers, meaning no two elements $X[i]$ and $X[j]$ are equal for $1 \leq i \neq j \leq N$.

a) Design a CRCW PRAM algorithm Sort($X[1 \dots N]$) with N^2 PUs, running in $O(1)$ time, that returns the sorted array $X'[1 \dots N]$, such that $X'[i] < X'[i+1]$ for all i. Specify the concurrency mode used for the concurrent writes.

b) What are its time, work and cost?

c) How can you make the algorithm work if the elements of $X[1 \dots N]$ are not unique?

d) How can you make the algorithm work on a CREW PRAM, still with N^2 PUs? What are its time, work and cost?

Exercise 10

Parallel matrix multiplication

Let $A[1 \dots N][1 \dots N]$ and $B[1 \dots N][1 \dots N]$ be two square matrices. Their product is the square matrix $C = AB$ whose elements are defined as follows:

$$C[i][j] = \sum_{k=1}^{N} A[i][k]\, B[k][j], \quad 1 \leq i, j \leq N$$

a) Design a CRCW PRAM algorithm MatMul(A, B) with N^3 PUs, running in $O(1)$ time, that carries out the multiplication of two $N \times N$ matrices $C = AB$, then returns the matrix C. What are its time, work and cost?

b) Do the same for a CREW PRAM, still with N^3 PUs. What are its time, work and cost? How can you make this algorithm work on an EREW PRAM as well?

Exercise 11

Simulation of concurrency modes on a CRCW PRAM

Consider a CRCW PRAM in *consistent* mode: simultaneous writes to the same memory cell are allowed only if they all write the same value.

a) Prove that a CRCW PRAM in consistent mode can simulate a CRCW PRAM in *fusion mode with* min (a cell written simultaneously by several PUs receives the minimum of the written values), with the same time up to a constant factor. The simulation may use more PUs than the simulated algorithm. Hint: with k^2 PUs, compare all the pairs among the k values written to a cell, and mark every value that is larger than another one; which concurrency mode does the marking need?

Exercise 12

Computing the depth of nodes in a tree

Let $parent[1 \dots N]$ be the array of parent pointers in a tree having N nodes (such that $parent[i] = \text{NIL}$ if the node i is the root of the tree, and $1 \leq parent[i] \neq i \leq N$ otherwise). The goal of this exercise is to compute the depth of all nodes in the tree in parallel. The depth of the root node is defined to be zero, and the depth of any other node is the depth of its parent plus one (i.e., $depth[i] = depth[parent[i]] + 1$).

a) Design a CREW PRAM algorithm TREEDEPTH($parent[1 \dots N]$) with N PUs to compute the depth of each node in the tree, and return it in an array. What are its time, work and cost? Would your algorithm work on an EREW PRAM?

b) This time, assume that the tree is binary, and in addition to parent pointers, each node i also has a pointer towards its left and right children $left[i]$ and $right[i]$ (with $left[i] = \text{NIL}$ if the node i has no left child, likewise for the right child). Design an EREW PRAM algorithm BINTREEDEPTH($parent[1 \dots N], left[1 \dots N], right[1 \dots N]$) with $O(N)$ PUs to compute and return the depth of all nodes in the tree in $O(\log_2 N)$ time. Hint: Consider the depth-first traversal of the tree, and try to use the prefix scan algorithm on this traversal.

Exercise 13

Parallel breadth-first search

The goal of this exercise is to perform a parallel breadth-first search (BFS) on a given directed graph $G(V, E)$ having $|V|$ vertices and $|E|$ edges $E[t] = (u_t, v_t)$ for $1 \leq t \leq |E|$, with $1 \leq u_t \neq v_t \leq |V|$. The graph is strongly connected: every vertex is reachable from every other one. Its *diameter* $\delta(G)$ is the largest distance between two vertices.

a) Design a CRCW PRAM algorithm BFS($G(V, E), s$) with $|E|$ PUs, one per edge, that performs a parallel BFS on G, starting from the vertex s ($1 \leq s \leq |V|$). The algorithm should compute and return two arrays: $parent[i]$, the parent of each vertex i in the BFS tree, and $level[i]$, the level of vertex i in the BFS tree (i.e., its distance from the starting vertex s). Specify the concurrency mode to be employed.

b) What are its time, work and cost, as functions of $|V|$, $|E|$ and $\delta(G)$?

Exercise 14

Connected components of an undirected graph

Let $G(V, E)$ be an undirected graph with $|V|$ vertices and $|E|$ edges $E[t] = (u_t, v_t)$ for $1 \leq t \leq |E|$, with $1 \leq u_t \neq v_t \leq |V|$. We want to label every vertex i with $C[i]$, the smallest vertex index of the connected component that i belongs to. We will use a simple algorithm: start with $C[i] = i$ for every vertex, then let every edge *merge* its two ends in parallel, by giving both of them the smaller of their two labels; repeat until no edge changes anything, i.e., until both ends of every edge carry the same label.

a) Design a CRCW PRAM algorithm CONNECTEDCOMPONENTS(V, E) that follows this idea, with $|E|$ PUs, one per edge. Specify the concurrency modes: for the writes to the labels, and for the test that stops the loop.

b) Show that the algorithm terminates and that the labels are correct in the end. Hint: let m be the smallest vertex of a component; after k iterations, which vertices of the component are guaranteed to carry the label m?

c) What are its time, work and cost, as functions of $|V|$, $|E|$ and the largest diameter D of a component (the diameter is the largest distance between two vertices of the component)? Is it work-optimal? Is it efficient?

d) Give a family of graphs with $|V|$ vertices on which the algorithm needs $\Theta(|V|)$ iterations. (The optimal $O(\log |V|)$ algorithms for this problem combine this idea with pointer jumping on the labels; they are beyond the scope of this course.)